PA 2 参考提示

数据结构课程团队

Contents

CST 2-1-2 Risk **2**

CST 2-1-3 Restore **3**

CST 2-2-2 Polynomial **4**

CST 2-2-3 Triangulation **5**

CST 2-3-2 Sect **6**

CST 2-3-1 & 2-3-3 Build **7**

CST 2-1-2　Risk

思路提示

首先，计算出每一天对应的风险基准值 d_i，由于考察区间的端点单调不降，因此等价于一个滑动窗口的最大值求解，可通过最大队列（讲义 04-J 节）实现。具体操作如下：当考察窗口滑动至第 i 天时，首先移除超过追溯日期的项。接着，从队尾开始，将所有对应值小于等于第 i 天新增确诊数的项移除，以维持队列的单调性。完成上述操作后，将第 i 项加入队尾。此时，队首所对应的值即为当前窗口的最大值 d_i。

在处理查询时，若对每组查询都遍历全部 d_i 值，总复杂度将达到 $O(Tn)$，无法满足时间要求。我们可以预先将所有 d_i 排序。对于每一组 p 和 q，可以通过二分查找来分别确定其位置，从而确定低、中风险的数量。

难点提示

请仔细阅读题面，确定滑动窗口区间端点的开闭。

请注意题目的数据范围，选用合适字长的数据类型。

CST 2-1-3　Restore

思路提示

首先考虑将这 n 个等差数列从小到大排列，那么第一个数列的第一个数一定为所有元素的最小值。

我们可以先确定第一个数列的首项，而公差 m 即为第一个数列的次项与首项之差，因此我们只需枚举所有可能的次项即相当于达成了枚举公差 m 的目的。由于第一个等差数列的所有项均为各个等差数列对应位置项的最小值，因此每个数列都有至少 $k-1$ 个数大于等于第一个数列的次项（即次项及以后的项）。于是第一个等差数列的次项只有可能是，所有元素排序后的第 2 小到第 $n+1$ 小的数，因此我们枚举公差的复杂度为 $O(n)$。

接下来，需要根据公差 m 尝试还原所有等差数列。我们可以根据第一个数列的首项，与公差，首先确定第一个数列的所有项，接着用类似双指针的思路进行 k 指针遍历还原每个数列的 k 项。具体来说，我们用 k 个指针分别指向第一个数列中的每项在所有元素排序后的位置，则第二个及以后的数列的第 i 项必然只能从第 i 个指针所指的位置开始往后寻找。我们在确定每一个新的数列的各项时，先通过第一个指针找到第一个未被使用的数（即为该数列的首项），基于首项我们便可以确定该数列的所有项，之后只需从小到大对于各个指针向后遍历尝试寻找首个未被使用的对应数值即可。在此过程中，如果未能找到对应的数值，则说明当前公差 m 不可行，可以直接跳出循环尝试下一个公差。

此外，我们可以证明最多可行解数量与 n 相关且远小于 n，因此实际运行效率会比理论复杂度分析结果更好。

难点提示

注意妥善处理相同元素。

CST 2-2-2　Polynomial

思路提示

本题要求实现多项式展开。其本质上就是中缀表达式的求值，可以按照 `04-D` 节介绍的方法来进行计算。

在实现中，可以使用数组存储系数。加减法对应系数的逐项相加减。乘法可以直接使用平方复杂度的竖式乘法。对于幂运算，由于题目限制最大次幂为 4，因此可以直接通过多次乘法展开。由于运算过程中的任何多项式最高次数不超过 64，单次加法和乘法的计算量是可控的（例如单次多项式乘法最坏情况为 32 次乘 32 次多项式）。

难点提示

如果遇到超时等问题，可以先在本地构造极端测试数据。一个典型的最坏情况是利用深度嵌套的幂运算生成一个高次项，例如 `x^4^4^4`，然后将这个高次项反复加减，例如 `x^4^4^4-x^4^4^4+x^4^4^4+...`，`(x+1)^4^4^4-(x+1)^4^4^4+...`。可以从以下方面着手优化：

- 避免过度使用 `new/delete`、避免不必要的内存复制，从而减小计算中间结果时的开销。
- 在计算乘法时，精确计算循环上下界，避免不必要的循环空转。为此可以记录此时最高项的次数。
- 减少不必要的取模，可以用加减法来避免一些取模操作。

如果遇到结果错误的问题，也可以考虑一些边界情况的构造，特别是数字解析、乘号省略、括号及括号嵌套、幂次以及它们的组合情况下程序是否能够正确运转。

CST 2-2-3　Triangulation

思路提示

本题要求实现对 x-单调多边形的三角剖分。整体只需按照题目中的算法描述进行。具体来说，首先将输入的简单多边形分解为一条上单调链和一条下单调链。这只需要找到 x 坐标最小（最左）和最大（最右）的两个顶点，它们即为上下两条链的分界点。

分解完成后，将多边形的所有顶点按 x 坐标从小到大排序（可根据单调性尝试直接归并，甚至一边归并一边扫描）。接着，依次扫描每个顶点，并使用一个栈来维护当前尚未完成剖分的优链。

处理新顶点时，需判断其属于上链还是下链，并与栈顶顶点进行比较。若为情况一，则依次把栈里面的点弹出。如果是情况二的话，则需要判断当前节点和栈中最顶端的两个点形成的角度，判断目前的栈顶是否是优节点。角度的判断可以转化为向量叉积的比较。当栈中顶点少于两个时，无法形成三角形，剖分过程结束。

假设向量 $\vec{a} = (x_a, y_a), \vec{b} = (x_b, y_b)$, 则叉积 $\vec{a} \times \vec{b} = x_a y_b - y_a x_b$，其正负性可以表征方向 $\vec{b}$ 在 $\vec{a}$ 的左手侧还是右手侧。

难点提示

计算叉积时注意是否会发生数据类型的溢出。

注意妥善处理端点时的边界情况，可以通过画图辅助理解一些边界情况是否可能发生。

CST 2-3-2　Sect

思路提示

首先考虑在一条链上要如何求解。对于每个元素，我们需要找到在其之前第一个大于它的元素，可以使用单调栈（讲义 04-I 节应用了单调栈）求解，每次新的元素达到，弹出栈中所有小于等于当前值的元素，此时栈顶即为对应结果，而后将该元素入栈。

对于树，由于每个分叉都有正统弟子的限制，实际上只有一个孩子节点需要继承分叉节点的单调栈。我们可以先通过一次遍历，计算子树的相应规模和以及继承单调栈的正统节点，相当于将树 “拆分” 为多个链，而后在每个链上进行一次单调栈计算即可。

难点提示

注意数据规模是否溢出。

拆分链的操作可以通过合理设置遍历顺序实现。注意链与链操作间对数据结构的重新初始化。

CST 2-3-1 & 2-3-3 Build

思路提示

题目的核心约束在于所有操作的总复杂度与路径查找长度（cost）相关，这意味着单次操作的效率必须与路径查询长度相关，而不能有全局的更新操作。

存储这棵树的时候，可以采用“左孩子右兄弟”表示法，也可以使用子树链表法，也即维护一个节点链表，代表某一个节点的所有子树。

对于子树的摘除和拼接，首先按照“通路表示法”确定节点，然后修改与父、兄节点的链接实现摘除和拼接。

对于子树规模的查询，可在每个节点上维护其对应子树的总规模。当一个子树被移动时，只需沿着其原父节点路径和新父节点路径，自底向上更新规模信息即可。

对于子树高度查询，若在更新时遍历所有子节点（即右兄弟链），复杂度会退化，不满足 cost 约束。解决方案是，在每个节点上额外存储其“右兄弟链后缀”上的最大高度。这样，当一个节点的子节点结构变化时，其新高度可以快速更新，从而将更新复杂度控制在 cost 范围内。

难点提示

在每一步操作时，都要保证路径上的子树规模/高度都被正确地更新。

请仔细阅读题面，确认“通路表示法”的边界条件（提示：在中途停止时依然需要将该行剩余的数从 `stdin` 中读入，避免干扰下次读取）。确认两次定位，移除和插入等操作的执行先后顺序。

注意题目中并不保证初始的树父亲编号一定小于孩子，在初始化读入时须留意。